10.4 友谊赛题解

花展（bloom）

部分分

测试点 1～4：枚举 $x \in [0, H-d]$，扫描所有预约，计算参加人数并统计最优日期数。时间复杂度为 $O(n(H-d+1))$。

测试点 5～8：人数只可能在 $l_i, r_i+1, l_i-d, r_i-d+1$ 处变化。取这些位置与 $0, H-d+1$，保留 $[0, H-d+1]$ 内的部分并排序去重。在相邻位置之间任选一个整数，扫描所有预约求出人数，再用这一段的整数个数更新方案数。时间复杂度为 $O(n^2)$。

正解

一份预约 $[l_i, r_i]$ 能参加花展，当且仅当 $x \in [l_i, r_i]$ 或 $x \in [l_i-d, r_i-d]$。因此，它对这两个区间的并集贡献 w_i，重叠部分只能计算一次。

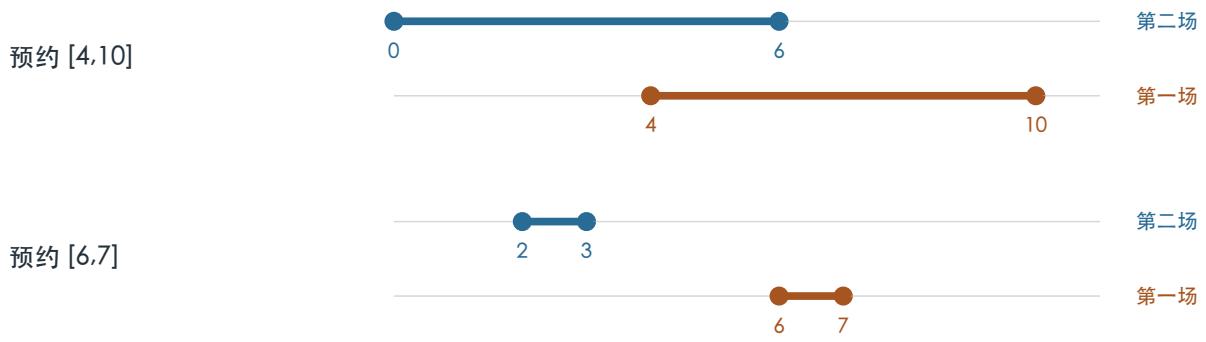


图 1　$d=4$。上例的区间可以合并，下例中间的空隙不能计入。

若 $r_i - d + 1 \geq l_i$，两个整数区间重叠或相邻，合并为 $[l_i-d, r_i]$；否则分别处理。再与合法范围 $[0, H-d]$ 取交集，忽略空区间。

对得到的每个带权区间 $[L, R]$，在 L 处加上权值，在 $R+1$ 处减去权值。将这些事件排序，同一坐标一起处理。处理完坐标 a 后，若下一个事件在 b，则 $[a, b-1]$ 内的参加人数都等于当前权值，共有 $b-a$ 个日期。遇到更大值时重置方案数，相等时累加即可。

额外在 $0, H-d+1$ 处加入权值变化为 0 的事件，保证所有合法日期都被统计，包括 $H=d$ 或没有预约能参加的情况。

单组时间复杂度 $O(n \log n)$，空间复杂度 $O(n)$；总时间复杂度 $O(\sum n \log n)$。

丝带（ribbon）

部分分

测试点 1 ~ 8：染色时逐格修改，询问时扫描所有不能再向两侧延长的同色段。长度为 $s \geq k$ 的一段贡献 $(s-k+1)(s-k+2)/2$ 个答案，累加即可。时间复杂度为 $O(nq)$。

特殊性质

特殊性质 A 中没有染色操作。预处理各长度的段数、长度和、长度平方和，再求后缀和，按正解的公式回答询问。时间复杂度为 $O(n+q)$。

正解

用有序集合按左端点维护所有极长同色段。染色 $[l,r]$ 时，先在 $r+1, l$ 处分割，删除 $[l,r]$ 中的整段，插入新段，再与相邻同色段合并。先分割右端，可以避免左端迭代器在第二次分割时失效。

初始至多有 n 段，每次染色只会新增常数段，而每个产生的段至多被删除一次。因此，全部操作涉及的段数为 $O(n+q)$，不需要对数据的随机性作任何假设。

对于一次询问，设长度至少为 k 的段数、长度和、长度平方和分别为 C, S_1, S_2。累加每段的贡献并展开，答案为

$$\frac{S_2 + (3-2k)S_1 + (k-1)(k-2)C}{2}.$$

用三个树状数组按段长维护这三个量。加入长度为 s 的段时，在下标 s 处分别加上 $1, s, s^2$；删除时减去。每次分割、合并也同步修改，询问则用前缀和相减求出 $[k,n]$ 的统计值。

时间复杂度为 $O((n+q)\log n)$，空间复杂度为 $O(n)$。

书架（shelf）

部分分

测试点 1 ~ 4：枚举类别的所有排列，逐本计算移动距离，取最大值作为代价，再统计最优代价及方案数。时间复杂度为 $O(nk!)$。

测试点 5 ~ 8：使用下文的两遍状态压缩 DP，但每次加入一类书时，逐本计算代价。每类被加入 2^{k-1} 次，时间复杂度为 $O((n+k)2^k)$。

正解

设某类书原来的位置为 $p_1 < \cdots < p_t$，前面已经放了 s 本书，则第 j 本的移动距离为 $|s-(p_j-j)|$。由于 $p_j - j$ 单调不降，最大距离只可能在首本或末本处取得。记类别 i 的数量、首末位置为 c_i, l_i, r_i，这一类的代价就是

$$cost(s,i) = \max(s+1-l_i,\ r_i-s-c_i).$$

设 $len_S = \sum_{i\in S} c_i$，$f_S$ 表示放好集合 S 中的类别后，已放书籍的最小可能最大移动距离。初始 $f_\varnothing = 0$，其余为无穷大。枚举下一类 $i \notin S$，有

$$f_{S\cup\{i\}} = \min\left(f_{S\cup\{i\}}, \max(f_S, cost(len_S, i))\right).$$

后续类别的位置只取决于 len_S，所以保留当前最小代价不会使答案变差。设全集为 U，最优代价为 $D = f_U$。

但不能在这遍 DP 中只统计达到 f_S 的前缀。当前较差的前缀，可能在后面出现更大距离后同样成为最优方案。例如序列 $1,2,3,1,1$，类别顺序 $(1,2)$、$(2,1)$ 的前缀代价分别为 $2,1$，最后接上类别 3 后却都得到最优代价 2。

因此，再设 g_S 表示放好 S 且所有距离均不超过 D 的排列数。初始 $g_\varnothing = 1$，其余为 0。对 $i \notin S$，若 $cost(len_S, i) \le D$，则

$$g_{S\cup\{i\}} \mathrel{+}= g_S.$$

所有加法对 10^9+7 取模。每个排列对应唯一的转移过程，而代价不超过 D 的排列恰好都是最优方案，故方案数为 g_U。

预处理各类书的信息和 len_S 后，单组时间复杂度为 $O(n+k2^k)$，空间复杂度为 $O(2^k+k)$；总时间复杂度为 $O(\sum(n+k2^k))$。

菜单（menu）

部分分

测试点 1～4：枚举订单排列，检查日期是否合法，将合法排列转成口味序列，在严格大于 b 的序列中取最小值。时间复杂度 $O(n \cdot n!)$，可以得到 20 分。

测试点 5～8：使用下文的贪心，但每天扫描全部剩余订单，重新计算 s_x 和可选口味。时间复杂度 $O(n^2)$，可以得到前 40 分。

特殊性质

特殊性质 A 中所有 $r_i = n$。任意一份已经可以开始的订单，都能与某个合法安排中今天的订单交换，不影响后续可行性。用可重复集合维护这些订单的口味，按正解的字典序贪心选择即可。时间复杂度 $O(n \log n)$，结合小数据做法可以得到 50 分。

正解

口味确定后，在该口味已经可以开始的订单中，总取截止日期最早的一份 u。若某个合法安排今天做同口味的 v，以后才做 u，交换它们仍然合法，且口味序列不变。因此，固定口味前缀后按此规则选订单，不会漏掉任何可行的后缀。

设今天是第 p 天，剩余订单仍然有解。对于 $x \geq p$，记

$$s_x = x - p + 1 - \#\{\text{剩余订单} i : r_i \leq x\}.$$

这些订单必须放在 $[p, x]$ 内，故 $s_x \geq 0$，且 $s_n = 0$。令 t 为第一个满足 $s_t = 0$ 的位置，则订单 i 今天可选，当且仅当 $l_i \leq p$ 且 $r_i \leq t$。

必要性在于 $[p, t]$ 恰好被截止日期不晚于 t 的订单占满，不能加入其他订单。对于充分性，任取一个合法安排，记第 u 天的订单截止日期为 d_u。将这份订单挪到 $v \in [u+1, d_u]$，再将原来在 v 的订单继续后移；只要不超过各自的截止日期，就不会违反开始日期限制。

每次都能到达 $[u+1, d_u]$ 内的所有位置，故从 p 出发能到达的位置构成一段 $[p, R]$。扩展停止时，其中所有订单的截止日期均不晚于 R，故 $s_R = 0$；又因为 $[p, t]$ 内的订单都不晚于 t 截止，扩展不能越过 t。由 t 的最小性，有 $R = t$。候选订单原来必在 $[p, t]$ 内，沿上述路径将订单依次后移，再将它放到 p，便得到合法安排。

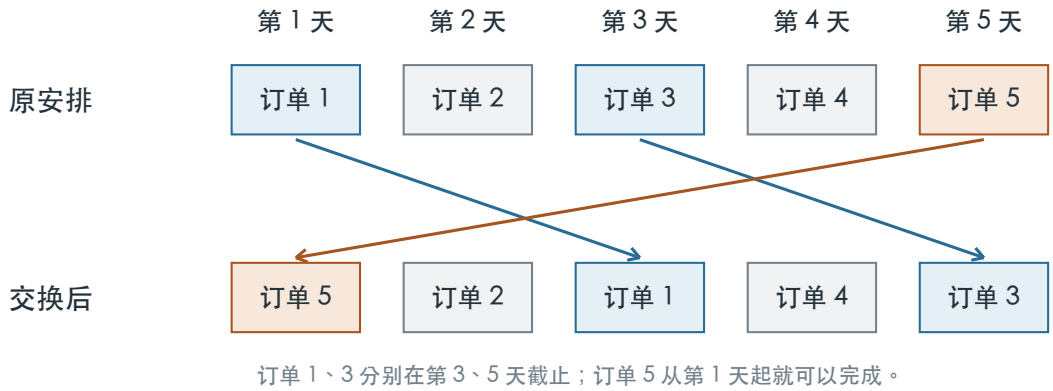


图 2　沿 $1 \to 3 \to 5$ 后移订单，再将订单 5 提前到第 1 天。

字典序后继与 b 第一次不同的位置应尽量靠后，该处选最小的更大口味，之后逐位取最小可行口味。先按 b 从左到右扫描，每天在选订单前，检查可选口味中是否有严格大于 b_p 的值，记录最后一个这样的位置及最小的更大口味。若不存在，输出 -1；否则重新扫描，保留此前的前缀，在该处增大，之后贪心补全。上述判定保证每次选择都有合法后续，同口味的选择也不会漏掉更优后缀，故能得到所求菜单。

初始 $s_x = x - \#\{i : r_i \le x\}$，用截止日期出现次数的前缀和求出。第 p 天完成截止日期为 r 的订单后，可用天数减少 1，而 $x \ge r$ 时被计数的订单数也减少 1，因此只需将 $[p+1, r-1]$ 减去 1。用线段树维护区间加和最小值，将用过的位置 p 设为无穷大，在树上找第一个零即可得到 t。

按开始日期加入订单，每种口味用小根堆维护已加入、未完成订单的截止日期，记堆顶为 d_c，空堆视为无穷大。口味 c 可选等价于 $d_c \le t$。再用一棵按口味编号排列的线段树维护 d_c 的区间最小值：查询不小于 k 的最小可选口味时，跳过编号全小于 k 或最小值大于 t 的区间，优先找左儿子。检查能否增大时取 $k = b_p + 1$，补全后缀时取 $k = 1$；选定后弹出堆顶并更新线段树。

每遍扫描中，每份订单各入堆、出堆一次。单组时间复杂度 $O(n \log n)$，空间复杂度 $O(n)$；总时间复杂度 $O(\sum n \log n)$。